Baladithya Balamurugan
1699864 bbalamur@ucsc.edu

CSE 102 (Fall 2020) – Lab 3

Before you begin the assignment, please read the following carefully.

- Read the Homework Guidelines.
- Every part of each question begins on a new page. Do not change this.
- This does not mean that you should write a full page for every question. Your answers should be short and precise. Lengthy and wordy answers will lose points.
- Do not change the format of this document. Simply type your answers as directed.
- You are **not** allowed to work in teams.
- You **are** allowed to discuss homework problems at a high level with groups of up to 4 classmates, however any submitted work must be your own. You may not share your written solutions with one another.
- Indicate any collaborators on the line below, and in each solution specify any key ideas obtained from group discussion.
- The lab is due for submission on Gradescope at the following times
  - All Sections: 17 November, 2020 (10 a.m.)

I have read and agree to the collaboration policy. – Baladithya Balamurugan, bbalamur@ucsc.edu
Collaborators:

Moving on a checkerboard.
Suppose that you are given an $n \times n$ checkerboard and a single checker. You must move the checker from the bottom (1^{st}) row of the board to the top (n^{th}) row of the board according to the following rule. At each step you may move the checker to one of three squares:

- the square immediately above,
- the square one up and one to the left (unless the checker is already in the leftmost column),
- the square one up and one right (unless the checker is already in the rightmost column).

Each time you move from square x to square y, you receive $p(x, y)$ dollars. The values $p(x, y)$ are known for all pairs (x, y) for which a move from x to y is legal. Note that $p(x, y)$ may be negative for some (x, y).

The end goal is to give an algorithm that determines a set of moves starting at the bottom row, and ending at the top row, and which gathers as many dollars as possible. Your algorithm is free to pick any square along the bottom row as a starting point, and any square along the top row as a destination in order to maximize the amount of money collected.

To that end, define $C[i, j]$ to be the maximum amount of money that can be collected in this process by moving a checker from any square on row 1, to the square at row i , column j. Obviously $C[1, j] = 0$ for all j in the range $1 \leq j \leq n$, since at least one move must be made to collect any money. Once the table entry $C[i, j]$ is known for all i and j such that $(1 \leq i \leq n, 1 \leq j \leq n)$, the maximum amount of money that can be collected by moving from row 1 to row n is computed using a transition function.

1. Observe that if one knows an optimal sequence of moves leading to square $y = (i, j)$, where $i > 1$, then the last move in that sequence must originate in one of the three neighboring squares in row $i-1$. These three squares have coordinates $(i - 1, j - 1)$ (if $j > 1$), $(i - 1, j)$ and $(i - 1, j + 1)$ (if $j < n$). Let that preceding square be denoted x. Prove the following.

   Claim: The subsequence of moves ending at x is itself an optimal sequence from row 1 to x.

   Solution.
   To prove the above claim we need to assume the following:

   (a) $S_y \leftarrow$ *an optimal sequence of moves to square* $y = (i, j)$

   (b) $S_x \leftarrow$ *a subsequence of* S_y *from row 1 to cell x. Optimal path:* $[1, x, y] = [S_x, y] = [S_y]$

   To prove the above claim we will use the proof of contradiction:

   Assume that subsequence S_x *is NOT optimal. By definition then there is a subsequence* S'_x *that is more optimal than* S_x *(more profit). Form this we can also say that there is a sequence* S'_y *(defined as* $[S'_x, y]$*) that is more optimal than* S_y *(more profit). This is a contradiction mainly because of the first assumtion we made. Since* S'_y *is a more optimal path* S_y *is no longer the most optimal which in turn is the contradiction. From this we say that out contradicting assumption is false and that* S_x *is an optimal subsequence of moves from row 1 to cell x.*

2. Find a recurrence relation to fill $C[i, j]$.

Solution.

$$x \in S = [(i-1, j-1), (i-1, j), (i-1, j+1)]$$

$$y = (i, j)$$

$$C[y] = \begin{cases} 0 & i = 1, 1 \leq j \leq n \\ max_x \left[C[x] + p(x, y)\right] & i > 1, 1 \leq j \leq n \end{cases}$$

Explanation *we start off as row 1 as 0 and that means whenever $i = 1$ the cost is 0. To find the most optimal path we need to maximize the moves therefore we choose the maximum of the possible moves that we can take to get closer to our end (there are 3 moves that are possible at each location). Since this is a down iteration algorithm we have our destination and from that we calculate our optimal path from row 1 by using the assumption that since there are 3 ways to move forward then there are also 3 ways to move back (only for the calculation not necessarily for the actual game). Given the destination we work backwards to find out most optimal path to row 1.*

3. It is now a simple matter to write an iterative algorithm to fill in the table C. We also wish to print out an optimal sequence of moves. So it is worthwhile to keep track of the predecessors as we fill in the table. Define $P[i, j]$ to be the predecessor of square (i, j) along an optimal sequence of moves starting in row 1, and ending at square (i, j), for $2 \leq i \leq n$ and $1 \leq j \leq n$. Write an algorithm, in pseudocode, to fill this table P. Describe, in a few sentences, what this algorithm does.

Algorithm 1 Optimal Path + Filling in Cost and Predecessor Array

```
1:  function CHECKERBOARDPATH(n,p)                          ▷ p - transition matrix
2:      C ← new Array(Array(n))                             ▷ nxn cost array
3:      P ← new Array(Array(n))                             ▷ nxn predecessor array
4:      C[1][:] ← 0                                         ▷ cost of row 1 is all 0
5:      for i=2; i≤n; i++ do
6:          for j=1; j≤n; j++ do
7:              if j == 0 then                              ▷ List of possible moves based on position in row
8:                  x ← [(i-1, j), (i-1, j+1)]              ▷ right edge
9:              else if j == (n-1) then
10:                 x ← [(i-1, j-1), (i-1, j)]              ▷ left edge
11:             else
12:                 x ← [(i-1, j-1), (i-1, j), (i-1, j+1)]  ▷ general case
13:             end if
14:             max ← −∞
15:             maxk ← -1
16:             for k=0; k<len(x); k++ do                   ▷ Find Max function
17:                 temp ← x[k] + p(x[k],(i,j))             ▷ calc cost using previous cost and transition matrix
18:                 if temp > max then
19:                     max ← temp
20:                     maxk ← x[k]
21:                 end if
22:             end for
23:             C[i][j] ← max
24:             P[i][j] ← maxk
25:         end for
26:     end for
27:     return C, P
28: end function
```

Solution.
The Algorithm spawns in the Cost matrix and Predecessor matrix and fills it going backwards from the destination to the start. C will be completely filled with the maximum profit needed to reach to any cell from row 1. As discussed before, each cell has 2-3 possible moves to go forwards which also means that there are 2-3 possible steps to get to the cell. The algorithm moves through each row and uses the previously calculated values to calculate the new values. The optimum values are calculated using the FindMax implementation. This fills out the entire C and P array with the optimal path to get to the cell selected and Returns the filled our matricies.

4. Now write an algorithm, again in pseudocode, that takes the table P filled by the last algorithm and prints the sequence.

Algorithm 2 Print Optimal Path

```
1:  function PRINTCHECKERBOARDPATH(C,P)
2:      max ← −∞
3:      maxcell ← (n,-1)
4:      n = len(C)
5:      for i=1; i ≤ len(x); i++ do                    ▷ Find Max function
6:          temp ← C[n][i]
7:          if temp > max then
8:              max ← temp
9:              maxcell ← (n,i)
10:         end if
11:     end for
12:     (predy,predx) ← maxcell                        ▷ y=row, x=column
13:     path ← List
14:     path.append(maxcell)
15:     while predy > 1 do
16:         pred ← P[predy][predx]                     ▷ gets the predecessor of the current cell
17:         path.append(pred)                          ▷ append the cell position to the path
18:         (predy,predx) ← pred
19:     end while
20:     for i=len(path); i > 0; i– do
21:         print(path[i])
22:     end for
23: end function
```

Solution.
The algorithm here first finds the max destination in the top row then proceeds to follow the predecessor path and append the data to a list that is printed out in the end.

5. Given a checkerboard, determine the run time to print an optimal sequence using the above two algorithms.

Solution.
Individual runtimes:

- *CheckerBoardPath(n,p)*
  The algorithm fills 2D arrays C and P using a nested for loop therefore the run time is $\Theta(n^2)$
- *printCheckerBoardPath(C,P)*
  The algorithm runs through a single row of size n to find the max ($\Theta(n)$*) then iterates down to the first row using the values in the predecessor array (*$\Theta(n)$*). This then results in a runtime of* $\Theta(n) + \Theta(n) \Rightarrow \Theta(n + n) \Rightarrow \Theta(n)$

Therefore the resulting runtime is as follows:

$$\Theta(n^2) + \Theta(n) \Rightarrow \Theta(n^2 + n) \Rightarrow \Theta(n^2)$$